

Deep Learning Technical Report

Skill Set Go AI/ML Internship — Week 2, Task 2.4 | Imangali Bayangaliyev

Problem Statement

This report evaluates PyTorch models on two supervised learning problems: predicting whether an individual's income exceeds \$50K/year from demographic and employment data, and classifying small color images into 10 object categories. A feedforward neural network was built for the first problem and a convolutional neural network (CNN) for the second; the CNN was then re-trained under four different hyperparameter configurations to study how optimizer, learning rate, and batch size affect performance under a fixed training budget.

Datasets

UCI Adult Income (~48,800 records, 1994 US Census data): a mix of numeric features (age, hours worked per week) and categorical features (education, occupation, workclass, native country), with a binary target of income $\leq \$50K$ or $> \$50K$. About 76% of records fall in the $\leq \$50K$ class, so a model that always predicts the majority class already scores 76% accuracy — this is the baseline the network must beat.

CIFAR-10: 60,000 32×32 RGB images split into 50,000 training / 10,000 test images, evenly balanced across 10 classes (e.g. airplane, automobile, bird, cat, dog). With 10 balanced classes, random guessing scores 10% accuracy, giving a much lower baseline to beat.

Architecture and Training Setup

Feedforward NN (Adult Income): categorical features were one-hot encoded and numeric features standardized, then passed through a fully-connected network with ReLU activations and a single logit output, trained with binary cross-entropy loss (BCEWithLogitsLoss) and the Adam optimizer.

CNN (CIFAR-10): two convolutional blocks (3→32 and 32→64 channels, each followed by ReLU and 2×2 max-pooling) feed into two fully-connected layers (128 hidden units → 10 output classes), trained with cross-entropy loss. The baseline configuration (Adam, learning rate 0.001, batch size 64) was trained for 10 epochs; the four hyperparameter-comparison runs in the next section each used 5 epochs so every configuration was trained under the same fixed budget.

Experiment Comparison

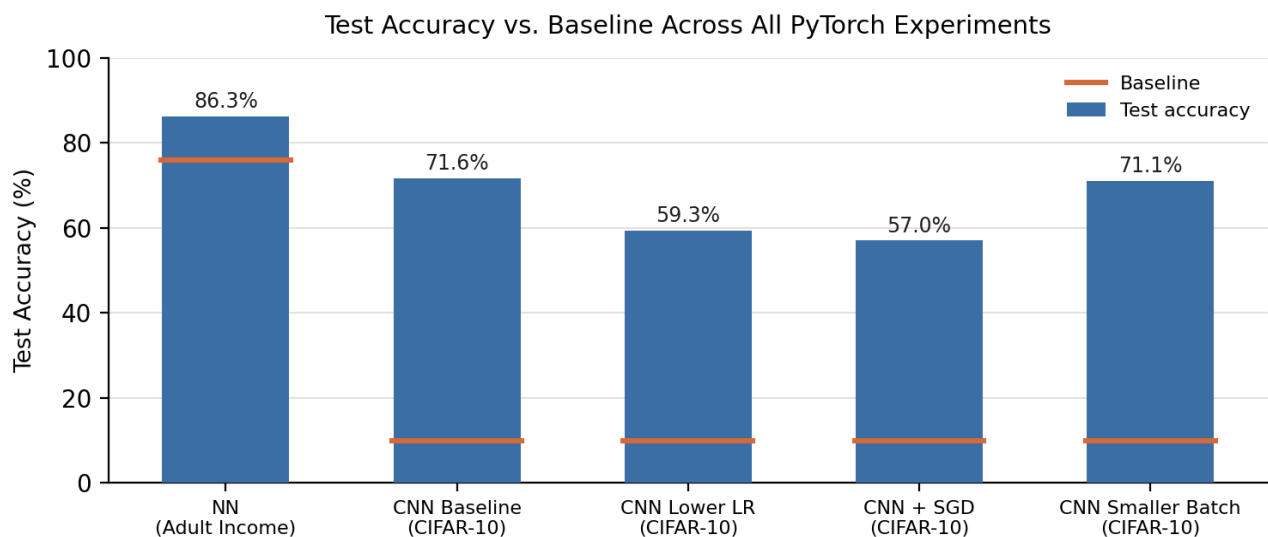


Figure 1. Test accuracy for each model/configuration (blue bars) against its task's baseline (orange line: 76% majority-class for the NN, 10% random-guess for all CNN runs).

Model / Experiment	Task	Optimizer	LR	Batch	Epochs	Test Acc.
Feedforward NN	Adult Income	Adam	default	-	~10	86.3%

CNN – Baseline	CIFAR-10	Adam	0.001	64	10	71.6%
CNN – Lower LR	CIFAR-10	Adam	0.0001	64	5	59.3%
CNN – SGD	CIFAR-10	SGD	0.001	64	5	57.0%
CNN – Smaller Batch	CIFAR-10	Adam	0.001	32	5	71.1%

Key Findings

- Both models substantially beat their respective baselines (NN: 86.3% vs. 76%; CNN: 71.6% vs. 10%), showing each learned real, generalizable patterns rather than exploiting class imbalance or guessing.
- Within a fixed 5-epoch budget, optimizer and learning rate had a much larger effect on CNN accuracy than batch size: dropping the learning rate 10× or switching to SGD each cost 12–15 points of accuracy, while halving the batch size cost under 1 point.
- The CNN's confusion matrix showed it reliably separates broad categories (e.g. vehicles vs. animals) but its single largest error is confusing cats with dogs — a known hard case due to their visual similarity, indicating the errors are concentrated in genuinely difficult cases rather than spread randomly.

Generalization Behavior

Feedforward NN: training loss kept falling while test loss plateaued after epoch 8 — mild overfitting, but the gap stayed small, so overall generalization is reasonable.

CNN baseline: the same divergence pattern appeared earlier, around epoch 4–5, suggesting this smaller image model overfits faster than the tabular network, plausibly because CIFAR-10 has higher input dimensionality relative to the amount of regularization used (none).

CNN hyperparameter variants: the Lower LR and SGD runs' weaker accuracy is more likely **underfitting** than overfitting — 5 epochs was probably too short for their slower updates to converge, not evidence that Adam is inherently the better optimizer long-term. This distinction matters: a lower score here reflects an undertrained model under this budget, not necessarily a worse one.

Limitations

- No systematic architecture search was done for either the NN or the CNN; both use a single hand-picked design.
- The CNN has no dropout, weight decay, or data augmentation, which likely contributes to its earlier overfitting.
- The hyperparameter comparison used only 5 epochs per run, which disadvantages slower-converging settings (SGD, lower LR) rather than reflecting their ceiling performance.
- Both tasks used a single train/test split with no cross-validation, so reported accuracies carry some run-to-run variance that wasn't measured.

Next Steps

- Re-run the SGD and Lower-LR configurations for more epochs (or with a learning-rate schedule) to see whether they close the gap with the Adam baseline once fully converged.
- Add dropout and/or simple data augmentation (random crop/flip) to the CNN to reduce overfitting and test whether it narrows the cat-vs-dog confusion specifically.
- Run a small learning-rate sweep around 0.001 for both models instead of relying on a single default value.
- Report per-class precision/recall for the CNN's weakest classes rather than overall accuracy alone.